Modul 6 — PHP & MySQL: Mengelola Seluruh Data Sekolah

Prasyarat: Modul 4 dan Modul 5 PHP.
Proyek: kelanjutan `crud_siswa`, database `db_sekolah`.
Target: PHP 8.0+, PDO MySQL, MySQL/MariaDB dari XAMPP.
Alokasi: 8 pertemuan × 3 JP.
Hasil: CRUD siswa, jurusan, mapel, dan nilai, pencarian, pagination, laporan, validasi relasi, serta dasar transaksi.

Gunakan data latihan dan backup database terlebih dahulu. Modul ini belum memiliki autentikasi; jalankan hanya di komputer lokal. Jangan membuka aplikasi ke internet sebelum menyelesaikan Modul 7 dan persiapan deployment yang sesuai.

1. Tujuan pembelajaran

Peserta mampu:

1. Memetakan relasi antarentitas menjadi form dan daftar data.
2. Memisahkan konfigurasi, repository, validasi, controller, dan tampilan.
3. Membuat CRUD empat entitas tanpa menerima nama tabel bebas dari pengguna.
4. Menggunakan prepared statement untuk nilai input dan allowlist untuk identifier SQL.
5. Menggunakan POST, token CSRF, serta Post/Redirect/Get untuk perubahan data.
6. Menangani email dan nama duplikat, foreign key tidak valid, serta nilai ganda.
7. Menjelaskan efek penghapusan terhadap data terkait.
8. Menyusun laporan JOIN dan transaksi yang dapat dibatalkan.

2. Hubungan dengan Modul 4

Kode Modul 4 tetap disimpan. Modul ini menambahkan panel `admin` agar siswa dapat membandingkan CRUD khusus dengan CRUD berbasis metadata. Nama folder `admin` belum berarti aksesnya sudah terlindungi.

```
C:\xampp\htdocs\crud_siswa\
├── koneksi.php                ← tetap dari Modul 4, menyediakan $pdo
├── index.php                  ← halaman lama tetap ada
├── detail.php
├── tambah.php
├── edit.php
├── hapus.php
└── admin\
    ├── bootstrap.php
```

```
    ├── helper.php
    ├── entitas.php
    ├── DataRepository.php
    ├── validasi.php
    └── index.php
```

Buat file sesuai urutan materi. Semua kode di bagian 5–10 adalah isi file lengkap. Jangan menggabungkan semua blok menjadi satu file. Buka panel melalui `http://localhost/crud_siswa/admin/index.php` setelah semua file selesai dibuat.

2.1 Pembagian tanggung jawab

File	Tanggung jawab
`bootstrap.php`	Memuat dependensi dan memulai session sebelum output
`helper.php`	Escape HTML, token CSRF, redirect
`entitas.php`	Allowlist tabel, primary key, label, field
`DataRepository.php`	Operasi SQL parametrik
`validasi.php`	Normalisasi dan aturan input
`index.php`	Mengarahkan aksi dan merender HTML

Controller masih digabung dengan view agar latihan dapat diikuti tanpa framework. Pada proyek lebih besar, keduanya dapat dipisahkan. CRUD generik di sini bukan alasan menghapus aturan khusus tiap entitas.

3. Memastikan struktur database

Database menggunakan struktur asli dari Modul DDL:

Entitas	Kolom utama	Aturan penting
jurusan	`id_jurusan`, `nama_jurusan`	Nama unik, maksimum 100 karakter
siswa	`id_siswa`, `nama`, `email`, `jenis_kelamin`, `tanggal_lahir`, `id_jurusan`, `created_at`	Email opsional dan unik; jurusan boleh NULL
mapel	`id_mapel`, `nama_mapel`	Nama unik, maksimum 100 karakter
nilai	`id_nilai`, `id_siswa`, `id_mapel`, `nilai_angka`, `semester`	FK siswa dan mapel; nilai desimal

Tidak ada kolom `nis`, `kelas`, atau `tahun_ajaran` pada skema dasar ini. Jangan memakai query contoh dari sistem lain tanpa migrasi.

Jalankan pemeriksaan berikut di phpMyAdmin:

```
USE db_sekolah;
SHOW CREATE TABLE siswa;
SHOW CREATE TABLE jurusan;
```

```
SHOW CREATE TABLE mapel;
SHOW CREATE TABLE nilai;
SHOW TABLE STATUS WHERE Name IN ('siswa', 'jurusan', 'mapel', 'nilai');
```

Pastikan engine tabel mendukung transaksi dan foreign key, umumnya InnoDB.

3.1 Kebijakan nilai

Untuk latihan ini:

- Nilai berada pada rentang 0–100 dengan maksimal dua angka desimal.
- Semester berupa nomor urut 1–6 selama masa belajar, bukan semester ganjil/genap yang diulang setiap tahun.
- Satu siswa memiliki paling banyak satu nilai akhir per mapel per semester.
- Jika aplikasi harus membedakan beberapa tahun ajaran atau jenis penilaian, model datanya perlu diperluas; jangan menumpuk data tersebut dalam skema ini.

3.2 Migrasi anti-duplikat tanpa menghapus data

Backup tabel `nilai`. Hentikan sementara penulisan data saat migrasi. Periksa data bermasalah:

```
SELECT id_siswa, id_mapel, semester, COUNT(*) AS jumlah
FROM nilai
GROUP BY id_siswa, id_mapel, semester
HAVING COUNT(*) > 1;

SELECT * FROM nilai
WHERE nilai_angka < 0 OR nilai_angka > 100
   OR semester < 1 OR semester > 6;

SHOW INDEX FROM nilai;
```

Jika ada duplikat, guru/operator harus menentukan nilai yang benar setelah membandingkan sumbernya. Jangan menjalankan penghapusan massal otomatis. Setelah data bersih, dan jika indeks setara belum ada, jalankan **sekali**:

```
ALTER TABLE nilai
ADD CONSTRAINT uq_nilai_siswa_mapel_semester
UNIQUE (id_siswa, id_mapel, semester);
```

`ALTER TABLE` pada MySQL dapat menyebabkan implicit commit; jangan menganggap migrasi DDL dapat dibatalkan seperti transaksi DML. Validasi aplikasi meningkatkan pesan kesalahan, sedangkan constraint unik tetap melindungi dari dua request yang masuk bersamaan.

3.3 Dampak penghapusan

Penghapusan	Perilaku FK asli
Jurusan	Siswa tetap ada; `id_jurusan` menjadi NULL

Penghapusan	Perilaku FK asli
Siswa	Semua nilai siswa ikut terhapus
Mapel	Semua nilai mapel ikut terhapus
Nilai	Hanya baris nilai tersebut terhapus

Panel akan menampilkan peringatan umum sebelum menghapus. Untuk sistem produksi, pertimbangkan arsip/soft delete dan audit trail sesuai kebijakan sekolah. Jangan mengubah CASCADE menjadi RESTRICT tanpa migrasi dan penyesuaian pesan aplikasi.

4. Kontrak koneksi

Gunakan `koneksi.php` dari Modul 4 yang menyediakan `$pdo`. Pastikan konfigurasi PDO menggunakan exception, fetch asosiatif, dan charset `utf8mb4` pada DSN.

Contoh atribut yang diperlukan, **bukan file koneksi tambahan**:

```
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
$pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
```

Jika koneksi gagal, tampilkan pesan umum dan catat detail di log. Jangan menampilkan username database, stack trace, atau password kepada pengguna. Akun database aplikasi produksi tidak seharusnya menggunakan `root`.

5. File `admin\helper.php`

```
<?php
declare(strict_types=1);

function eAdmin(mixed $value): string
{
    return htmlspecialchars((string) $value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}

function csrfToken(): string
{
    if (!isset($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

function verifyCsrf(): void
{
    $token = $_POST['csrf_token'] ?? null;
```

```
    if (!is_string($token) || !isset($_SESSION['csrf_token'])
        || !hash_equals($_SESSION['csrf_token'], $token)) {
        http_response_code(403);
        exit('Token formulir tidak valid. Muat ulang halaman.');
    }
}

function redirectAdmin(string $url): void
{
    header('Location: ' . $url, true, 303);
    exit;
}

function teksInput(array $input, string $key): string
{
    $value = $input[$key] ?? '';
    if (!is_string($value)) {
        throw new InvalidArgumentException('Bentuk input tidak valid: ' . $key);
    }
    return trim($value);
}

function idPositif(string $value): int
{
    $id = filter_var($value, FILTER_VALIDATE_INT, ['options' => ['min_range' => 1]]);
    if ($id === false) {
        throw new InvalidArgumentException('ID harus berupa bilangan bulat positif.');
    }
    return $id;
}
```

Escape HTML dilakukan ketika menampilkan data, bukan sebelum menyimpan ke database. Prepared statement melindungi konteks SQL; `htmlspecialchars()` melindungi konteks teks/atribut HTML. Keduanya tidak saling menggantikan.

6. File `admin\entitas.php`

```
<?php
declare(strict_types=1);

return [
    'siswa' => [
        'pk' => 'id_siswa', 'label' => 'Siswa', 'cari' => 'nama',
        'fields' => [
            'nama' => 'Nama', 'email' => 'Email',
            'jenis_kelamin' => 'Jenis kelamin',
            'tanggal_lahir' => 'Tanggal lahir', 'id_jurusan' => 'Jurusan',
        ],
    ],
```

```
    'jurusan' => [
        'pk' => 'id_jurusan', 'label' => 'Jurusan', 'cari' => 'nama_jurusan',
        'fields' => ['nama_jurusan' => 'Nama jurusan'],
    ],
    'mapel' => [
        'pk' => 'id_mapel', 'label' => 'Mata pelajaran', 'cari' => 'nama_mapel',
        'fields' => ['nama_mapel' => 'Nama mata pelajaran'],
    ],
    'nilai' => [
        'pk' => 'id_nilai', 'label' => 'Nilai', 'cari' => 'semester',
        'fields' => [
            'id_siswa' => 'Siswa', 'id_mapel' => 'Mata pelajaran',
            'nilai_angka' => 'Nilai angka', 'semester' => 'Semester',
        ],
    ],
];
```

Nama tabel dan kolom berasal dari kode tetap, bukan langsung dari request. Placeholder tidak dapat digunakan untuk nama tabel, nama kolom, atau kata `ASC` / `DESC`. Karena itu identifier wajib dipilih dari allowlist.

7. File `admin\DataRepository.php`

```
<?php
declare(strict_types=1);

final class DataRepository
{
    private PDO $pdo;
    private array $config;

    public function __construct(PDO $pdo, array $config)
    {
        $this->pdo = $pdo;
        $this->config = $config;
    }

    private function aturan(string $entitas): array
    {
        if (!isset($this->config[$entitas])) {
            throw new InvalidArgumentException('Entitas tidak tersedia.');
        }
        return $this->config[$entitas];
    }

    public function hitung(string $entitas, string $q): int
    {
        $c = $this->aturan($entitas);
        $stmt = $this->pdo->prepare(
            "SELECT COUNT(*) FROM `$entitas` WHERE `{$c['cari']}` LIKE :q"
```

```
        );
        $stmt->execute(['q' => '%' . $q . '%']);
        return (int) $stmt->fetchColumn();
    }

    public function daftar(string $entitas, string $q, int $limit, int $offset): array
    {
        $c = $this->aturan($entitas);
        $stmt = $this->pdo->prepare(
            "SELECT * FROM `$entitas` WHERE `{$c['cari']}` LIKE :q
             ORDER BY `{$c['pk']}` DESC LIMIT :batas OFFSET :offset"
        );
        $stmt->bindValue(':q', '%' . $q . '%', PDO::PARAM_STR);
        $stmt->bindValue(':batas', $limit, PDO::PARAM_INT);
        $stmt->bindValue(':offset', $offset, PDO::PARAM_INT);
        $stmt->execute();
        return $stmt->fetchAll();
    }

    public function satu(string $entitas, int $id): array|false
    {
        $c = $this->aturan($entitas);
        $stmt = $this->pdo->prepare("SELECT * FROM `$entitas` WHERE `{$c['pk']}` = :id");
        $stmt->execute(['id' => $id]);
        return $stmt->fetch();
    }

    public function simpan(string $entitas, ?int $id, array $data): void
    {
        $c = $this->aturan($entitas);
        $kolom = array_keys($c['fields']);
        $parameter = [];
        foreach ($kolom as $nama) {
            $parameter[$nama] = $data[$nama];
        }
        if ($id === null) {
            $namaKolom = implode(', ', array_map(fn ($k) => "`$k`", $kolom));
            $placeholder = implode(', ', array_map(fn ($k) => ':' . $k, $kolom));
            $sql = "INSERT INTO `$entitas` ($namaKolom) VALUES ($placeholder)";
        } else {
            $set = implode(', ', array_map(fn ($k) => "`$k` = :$k", $kolom));
            $sql = "UPDATE `$entitas` SET $set WHERE `{$c['pk']}` = :id";
            $parameter['id'] = $id;
        }
        $this->pdo->prepare($sql)->execute($parameter);
    }

    public function hapus(string $entitas, int $id): void
    {
        $c = $this->aturan($entitas);
        $stmt = $this->pdo->prepare("DELETE FROM `$entitas` WHERE `{$c['pk']}` = :id");
```

```
        $stmt->execute(['id' => $id]);
    }

    public function pilihan(string $entitas): array
    {
        $c = $this->aturan($entitas);
        if (!in_array($entitas, ['siswa', 'jurusan', 'mapel'], true)) {
            throw new InvalidArgumentException('Pilihan tidak tersedia.');
        }
        $stmt = $this->pdo->query(
            "SELECT `{$c['pk']}` AS id, `{$c['cari']}` AS nama
             FROM `$entitas` ORDER BY `{$c['cari']}`, `{$c['pk']}`"
        );
        return $stmt->fetchAll();
    }
}
```

7.1 Membaca pola repository

- `aturan()` memastikan semua operasi memakai entitas terdaftar.
- `simpan()` menyalin hanya field yang diizinkan, sehingga input tambahan tidak otomatis menjadi kolom SQL.
- `LIMIT` dan `OFFSET` di-bind sebagai integer; ini penting terutama dengan native prepared statement.
- Pencarian menggunakan `LIKE`. Karakter `%` dan `_` berfungsi sebagai wildcard; pada modul ini perilaku itu diperbolehkan dan bukan SQL injection.
- `rowCount()` tidak dipakai untuk menentukan apakah update berhasil: MySQL dapat mengembalikan nol ketika data baru sama dengan data lama.
- Pilihan dropdown memuat semua data induk agar tutorial sederhana. Pada data besar gunakan endpoint pencarian terautentikasi dan pagination, bukan ribuan option sekaligus.

8. File `admin\validasi.php`

```
<?php
declare(strict_types=1);

function validasiData(string $entitas, array $input, DataRepository $repo): array
{
    $namaValid = static function (string $nilai, string $label): string {
        $panjang = preg_match_all('/./us', $nilai);
        if ($nilai === '' || $panjang === false || $panjang > 100) {
            throw new InvalidArgumentException($label . ' wajib diisi, maksimal 100 karakter.');
        }
        return $nilai;
    };
    $relasi = static function (string $target, string $nilai) use ($repo): int {
        $id = idPositif($nilai);
        if ($repo->satu($target, $id) === false) {
            throw new InvalidArgumentException('Data ' . $target . ' tidak ditemukan.');
```

```
        }
        return $id;
    };

    if ($entitas === 'jurusan' || $entitas === 'mapel') {
        $kolom = $entitas === 'jurusan' ? 'nama_jurusan' : 'nama_mapel';
        return [$kolom => $namaValid(teksInput($input, $kolom), 'Nama')];
    }

    if ($entitas === 'siswa') {
        $nama = $namaValid(teksInput($input, 'nama'), 'Nama siswa');
        $email = teksInput($input, 'email');
        if ($email !== '' && (strlen($email) > 150
            || filter_var($email, FILTER_VALIDATE_EMAIL) === false)) {
            throw new InvalidArgumentException('Email tidak valid atau terlalu panjang.');
        }
        $jk = teksInput($input, 'jenis_kelamin');
        if (!in_array($jk, ['L', 'P'], true)) {
            throw new InvalidArgumentException('Jenis kelamin harus L atau P.');
        }
        $tanggal = teksInput($input, 'tanggal_lahir');
        if ($tanggal !== '') {
            $date = DateTimeImmutable::createFromFormat('!Y-m-d', $tanggal);
            if (!$date || $date->format('Y-m-d') !== $tanggal
                || $tanggal < '1000-01-01' || $tanggal > date('Y-m-d')) {
                throw new InvalidArgumentException('Tanggal lahir tidak valid atau di masa depan.');
            }
        }
        $jurusan = teksInput($input, 'id_jurusan');
        return [
            'nama' => $nama, 'email' => $email === '' ? null : $email,
            'jenis_kelamin' => $jk, 'tanggal_lahir' => $tanggal === '' ? null : $tanggal,
            'id_jurusan' => $jurusan === '' ? null : $relasi('jurusan', $jurusan),
        ];
    }

    if ($entitas === 'nilai') {
        $angka = teksInput($input, 'nilai_angka');
        if (!preg_match('/^\d{1,3}(\.\d{1,2})?$/D', $angka) || (float) $angka > 100) {
            throw new InvalidArgumentException('Nilai harus 0–100, maksimal dua desimal dengan titik.');
        }
        $semester = filter_var(teksInput($input, 'semester'), FILTER_VALIDATE_INT,
            ['options' => ['min_range' => 1, 'max_range' => 6]]);
        if ($semester === false) {
            throw new InvalidArgumentException('Semester harus bilangan bulat 1–6.');
        }
        return [
            'id_siswa' => $relasi('siswa', teksInput($input, 'id_siswa')),
            'id_mapel' => $relasi('mapel', teksInput($input, 'id_mapel')),
            'nilai_angka' => $angka, 'semester' => $semester,
        ];
```

```
    }
    throw new InvalidArgumentException('Entitas tidak valid.');
}
```

8.1 Mengapa validasi dilakukan di server?

Atribut HTML seperti `required`, `min`, dan `max` membantu pengalaman pengguna, tetapi request dapat dikirim tanpa browser. Server tetap memeriksa:

- Bentuk input: string, bukan array.
- Panjang dan format.
- Rentang nilai dan semester.
- Keberadaan data induk.
- Kolom yang diperbolehkan.

Pengecekan relasi sebelum menyimpan tidak menghilangkan kemungkinan data induk dihapus oleh request lain sesudah pengecekan. Foreign key database tetap menjadi perlindungan terakhir; exception tetap harus ditangani.

9. File `admin\bootstrap.php`

```
<?php
declare(strict_types=1);

require_once dirname(__DIR__) . '/koneksi.php';
require_once __DIR__ . '/helper.php';
require_once __DIR__ . '/DataRepository.php';
require_once __DIR__ . '/validasi.php';

if (session_status() !== PHP_SESSION_ACTIVE) {
    session_start();
}

$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
$pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
$config = require __DIR__ . '/entitas.php';
$repo = new DataRepository($pdo, $config);
```

Session pada tahap ini dipakai untuk token CSRF dan pesan setelah redirect, bukan untuk login. Semua pemuatan dilakukan sebelum HTML agar cookie dan header dapat dikirim.

10. File `admin\index.php`

```
<?php
declare(strict_types=1);
require_once __DIR__ . '/bootstrap.php';
```

```
try {
    $entitas = teksInput($_GET, 'entitas') ?: 'siswa';
    $aksi = teksInput($_GET, 'aksi') ?: 'daftar';
    if (!isset($config[$entitas]) || !in_array($aksi, ['daftar', 'form', 'simpan', 'hapus'], true)) {
        http_response_code(404);
        exit('Halaman tidak ditemukan.');
    }
    // AUTH_ROLE_CHECK: Modul 7 menambahkan pemeriksaan hak akses di sini.
    $c = $config[$entitas];
    $url = 'index.php?entitas=' . urlencode($entitas);
    $error = '';
    $data = [];
    $id = null;
    $method = $_SERVER['REQUEST_METHOD'];

    if (in_array($aksi, ['simpan', 'hapus'], true)) {
        if ($method !== 'POST') {
            header('Allow: POST');
            http_response_code(405);
            exit('Gunakan POST untuk perubahan data.');
        }
        verifyCsrf();
        $idTeks = teksInput($_POST, 'id');
        $id = $idTeks === '' ? null : idPositif($idTeks);
        if (($aksi === 'hapus' && $id === null)
            || ($id !== null && $repo->satu($entitas, $id) === false)) {
            http_response_code(404);
            exit('Data tidak ditemukan.');
        }
        try {
            if ($aksi === 'hapus') {
                $repo->hapus($entitas, $id);
            } else {
                $data = validasiData($entitas, $_POST, $repo);
                $repo->simpan($entitas, $id, $data);
            }
            $_SESSION['pesan_admin'] = 'Perubahan berhasil disimpan.';
            redirectAdmin($url);
        } catch (InvalidArgumentException $e) {
            $error = $e->getMessage();
            http_response_code(422);
        } catch (PDOException $e) {
            $kode = (int) ($e->errorInfo[1] ?? 0);
            $error = $kode === 1062
                ? 'Data duplikat: periksa email, nama, atau kombinasi siswa–mapel–semester.'
                : ($kode === 1451 || $kode === 1452
                    ? 'Relasi data berubah atau tidak valid. Muat ulang dan periksa pilihan.'
                    : 'Penyimpanan gagal. Hubungi pengelola aplikasi.');
            if (!in_array($kode, [1062, 1451, 1452], true)) {
                error_log('CRUD gagal, kode database: ' . $kode);
```

```
            }
            http_response_code(in_array($kode, [1062, 1451, 1452], true) ? 409 : 500);
        }
        $aksi = $aksi === 'hapus' ? 'daftar' : 'form';
        foreach ($c['fields'] as $field => $label) {
            $data[$field] = is_string($_POST[$field] ?? null) ? $_POST[$field] : '';
        }
    } elseif ($method !== 'GET') {
        header('Allow: GET');
        http_response_code(405);
        exit('Metode tidak tersedia.');
    } elseif ($aksi === 'form') {
        $idTeks = teksInput($_GET, 'id');
        if ($idTeks !== '') {
            $id = idPositif($idTeks);
            $data = $repo->satu($entitas, $id);
            if ($data === false) {
                http_response_code(404);
                exit('Data tidak ditemukan.');
            }
        }
    }

    $pilihan = [];
    foreach (['siswa', 'jurusan', 'mapel'] as $target) {
        $pilihan['id_' . $target] = $repo->pilihan($target);
    }
    $labelRelasi = [];
    foreach ($pilihan as $field => $items) {
        $labelRelasi[$field] = array_column($items, 'nama', 'id');
    }
    $q = teksInput($_GET, 'q');
    $halamanTeks = teksInput($_GET, 'halaman');
    $halaman = $halamanTeks === '' ? 1 : idPositif($halamanTeks);
    $total = $repo->hitung($entitas, $q);
    $maksHalaman = max(1, (int) ceil($total / 10));
    $halaman = min($halaman, $maksHalaman);
    $baris = $repo->daftar($entitas, $q, 10, ($halaman - 1) * 10);
    $pesan = $_SESSION['pesan_admin'] ?? '';
    unset($_SESSION['pesan_admin']);
} catch (InvalidArgumentException $e) {
    http_response_code(400);
    exit(eAdmin($e->getMessage()));
} catch (PDOException $e) {
    error_log('Pembacaan panel gagal; SQLSTATE: ' . $e->getCode());
    http_response_code(500);
    exit('Data tidak dapat dimuat. Periksa layanan database.');
}
?>
<!doctype html>
<html lang="id">
```

```
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Kelola <?= eAdmin($c['label']) ?></title>
    <style>
        body { font: 16px system-ui; max-width: 1100px; margin: 2rem auto; padding: 0 1rem; }
        nav a { margin-right: 1rem; }
        table { border-collapse: collapse; width: 100%; }
        td, th { border: 1px solid #bbb; padding: .5rem; text-align: left; }
        label { display: block; margin: .8rem 0; }
        input, select, button { font: inherit; padding: .35rem; }
        .error { color: #a00000; } .pesan { color: #075c20; }
        .scroll { overflow-x: auto; } .inline { display: inline; }
    </style>
</head>
<body>
<nav aria-label="Data sekolah">
<?php foreach ($config as $key => $item): ?>
    <a href="index.php?entitas=<?= eAdmin($key) ?>"><?= eAdmin($item['label']) ?></a>
<?php endforeach; ?>
</nav>
<h1>Kelola <?= eAdmin($c['label']) ?></h1>
<?php if ($pesan !== ''): ?><p class="pesan" role="status"><?= eAdmin($pesan) ?></p><?php endif; ?>
<?php if ($error !== ''): ?><p class="error" role="alert"><?= eAdmin($error) ?></p><?php endif; ?>

<?php if ($aksi === 'form'): ?>
<h2><?= $id === null ? 'Tambah' : 'Edit' ?> data</h2>
<form method="post" action="<?= eAdmin($url . '&aksi=simpan') ?>">
    <input type="hidden" name="csrf_token" value="<?= eAdmin(csrfToken()) ?>">
    <input type="hidden" name="id" value="<?= eAdmin($id ?? '') ?>">
    <?php foreach ($c['fields'] as $field => $label): ?>
    <label><?= eAdmin($label) ?>
    <?php if (isset($pilihan[$field])): ?>
        <select name="<?= eAdmin($field) ?>" <?= $field === 'id_jurusan' ? '' : 'required' ?>>
            <option value="">— Pilih —</option>
            <?php foreach ($pilihan[$field] as $option): ?>
            <option value="<?= eAdmin($option['id']) ?>"
                <?= (string) ($data[$field] ?? '') === (string) $option['id'] ? 'selected' : '' ?>>
                <?= eAdmin($option['nama'] . ' (#' . $option['id'] . ')') ?>
            </option>
            <?php endforeach; ?>
        </select>
    <?php elseif ($field === 'jenis_kelamin'): ?>
        <select name="jenis_kelamin" required>
            <option value="">— Pilih —</option>
            <?php foreach (['L' => 'Laki-laki', 'P' => 'Perempuan'] as $key => $labelJk): ?>
            <option value="<?= eAdmin($key) ?>" <?= ($data[$field] ?? '') === $key ? 'selected' : '' ?>>
                <?= eAdmin($labelJk) ?>
            </option>
            <?php endforeach; ?>
        </select>
```

```
    <?php else:
        $type = $field === 'email' ? 'email' : ($field === 'tanggal_lahir' ? 'date'
            : (in_array($field, ['nilai_angka', 'semester'], true) ? 'number' : 'text'));
    ?>
        <input name="<?= eAdmin($field) ?>" type="<?= eAdmin($type) ?>"
            value="<?= eAdmin($data[$field] ?? '') ?>"
            <?= in_array($field, ['email', 'tanggal_lahir'], true) ? '' : 'required' ?>
            <?= $field === 'nilai_angka' ? 'min="0" max="100" step="0.01"' : '' ?>
            <?= $field === 'semester' ? 'min="1" max="6" step="1"' : '' ?>
            <?= $type === 'text' ? 'maxlength="100"' : '' ?>
            <?= $field === 'email' ? 'maxlength="150"' : '' ?>>
    <?php endif; ?>
    </label>
    <?php endforeach; ?>
    <button type="submit">Simpan</button>
    <a href="<?= eAdmin($url) ?>">Batal</a>
</form>
<?php else: ?>
<p><a href="<?= eAdmin($url . '&aksi=form') ?>">Tambah <?= eAdmin($c['label']) ?></a></p>
<form method="get">
    <input type="hidden" name="entitas" value="<?= eAdmin($entitas) ?>">
    <label>Cari <?= $entitas === 'nilai' ? 'semester' : 'nama' ?>
        <input name="q" value="<?= eAdmin($q) ?>">
    </label>
    <button type="submit">Cari</button>
</form>
<p><?= eAdmin($total) ?> data. Halaman <?= eAdmin($halaman) ?> dari <?= eAdmin($maksHalaman) ?>.</p>
<p>Perhatian: hapus siswa/mapel juga menghapus nilai terkait; hapus jurusan mengosongkan jurusan siswa.</p>
<div class="scroll">
<table>
<thead><tr><th>ID</th>
<?php foreach ($c['fields'] as $label): ?><th><?= eAdmin($label) ?></th><?php endforeach; ?>
<th>Aksi</th></tr></thead>
<tbody>
<?php foreach ($baris as $row): ?>
<tr><td><?= eAdmin($row[$c['pk']]) ?></td>
<?php foreach ($c['fields'] as $field => $label): ?>
<td><?= eAdmin(isset($labelRelasi[$field])
    ? ($labelRelasi[$field][$row[$field] ?? ''] ?? '—') : ($row[$field] ?? '—')) ?></td>
<?php endforeach; ?>
<td>
<a href="<?= eAdmin($url . '&aksi=form&id=' . $row[$c['pk']]) ?>">Edit</a>
<form class="inline" method="post" action="<?= eAdmin($url . '&aksi=hapus') ?>"
    onsubmit="return confirm('Hapus data beserta dampak relasinya?')">
    <input type="hidden" name="csrf_token" value="<?= eAdmin(csrfToken()) ?>">
    <input type="hidden" name="id" value="<?= eAdmin($row[$c['pk']]) ?>">
    <button type="submit">Hapus</button>
</form>
</td></tr>
<?php endforeach; ?>
<?php if ($baris === []): ?><tr><td colspan="<?= count($c['fields']) + 2 ?>">Data tidak ditemukan.</td></tr><?php endif;
```

```
?>
</tbody>
</table>
</div>
<nav aria-label="Halaman daftar">
<?php if ($halaman > 1): ?>
<a href="<?= eAdmin($url . '&' . http_build_query(['q' => $q, 'halaman' => $halaman - 1])) ?>">Sebelumnya</a>
<?php endif; ?>
<?php if ($halaman < $maksHalaman): ?>
<a href="<?= eAdmin($url . '&' . http_build_query(['q' => $q, 'halaman' => $halaman + 1])) ?>">Berikutnya</a>
<?php endif; ?>
</nav>
<?php endif; ?>
</body>
</html>
```

10.1 Mengikuti satu request sampai selesai

Tambah siswa:

1. GET `?entitas=siswa&aksi=form` menampilkan form.
2. Browser POST ke `?entitas=siswa&aksi=simpan`.
3. Controller memeriksa metode dan token CSRF.
4. ID kosong berarti INSERT; ID terisi berarti UPDATE.
5. Validator memeriksa input dan relasi.
6. Repository menyimpan field yang diizinkan.
7. Pesan disimpan ke session; redirect 303 menuju daftar.
8. Refresh daftar tidak mengulangi POST sebelumnya.

Edit nilai:

1. ID pada URL divalidasi sebagai integer positif.
2. Baris dibaca dari tabel `nilai`.
3. Form menampilkan siswa, mapel, angka, dan semester saat ini.
4. Saat POST, server kembali memeriksa ID dan seluruh input.
5. Constraint unik mencegah kombinasi yang sudah digunakan baris lain.

Hapus: GET tidak boleh menghapus data. Dialog JavaScript hanya membantu pengguna; perlindungan server tetap POST + CSRF + pemeriksaan ID, kemudian otorisasi pada Modul 7.

11. Praktik terarah empat entitas

11.1 Jurusan

1. Tambahkan jurusan baru dengan nama unik.
2. Edit namanya tanpa mengubah ID.
3. Coba nama kosong dan nama yang sudah ada.

4. Kaitkan satu siswa ke jurusan tersebut.
5. Hapus jurusan pada data latihan; siswa harus tetap ada dengan jurusan kosong.

11.2 Siswa

1. Tambahkan siswa dengan email kosong dan jurusan kosong.
2. Tambahkan siswa lain dengan email kosong; keduanya seharusnya berhasil.
3. Isi email yang sama pada dua siswa; penyimpanan kedua harus ditolak.
4. Coba tanggal `2025-02-30` melalui request manual; harus ditolak server.
5. Edit tanpa mengubah apa pun; tetap merupakan operasi yang sah.

11.3 Mata pelajaran

1. Tambahkan mapel, misalnya `Pemrograman Web Lanjutan`.
2. Pastikan nama unik.
3. Tambahkan nilai yang merujuk mapel tersebut.
4. Hapus mapel hanya pada data latihan; nilai terkait ikut terhapus.

11.4 Nilai

1. Pilih siswa dan mapel yang sudah ada.
2. Simpan nilai `0`, `75.50`, dan `100` pada kombinasi berbeda.
3. Tolak `-1`, `100.01`, `abc`, dan `75.123`.
4. Tolak semester `0`, `7`, dan `1.5`.
5. Simpan kombinasi siswa–mapel–semester yang sama dua kali; percobaan kedua ditolak.
6. Edit baris lama tanpa mengubah kombinasinya; seharusnya tetap berhasil.

12. Laporan dengan JOIN dan agregasi

Bagian ini dijalankan di phpMyAdmin sebagai latihan SQL. Tampilan rapor siswa dari `detail.php` Modul 4 tetap dapat digunakan dengan data baru.

12.1 Daftar nilai dengan nama, bukan hanya ID

```
SELECT n.id_nilai, s.nama AS nama_siswa,
       j.nama_jurusan, m.nama_mapel, n.semester, n.nilai_angka
FROM nilai n
JOIN siswa s ON s.id_siswa = n.id_siswa
JOIN mapel m ON m.id_mapel = n.id_mapel
LEFT JOIN jurusan j ON j.id_jurusan = s.id_jurusan
ORDER BY s.nama, n.semester, m.nama_mapel;
```

`LEFT JOIN jurusan` mempertahankan siswa tanpa jurusan. Mengganti dengan `INNER JOIN` akan menghilangkan baris tersebut dari laporan.

12.2 Rata-rata per siswa, termasuk yang belum memiliki nilai

```
SELECT s.id_siswa, s.nama,
       COUNT(n.id_nilai) AS jumlah_nilai,
       ROUND(AVG(n.nilai_angka), 2) AS rata_rata
FROM siswa s
LEFT JOIN nilai n ON n.id_siswa = s.id_siswa
GROUP BY s.id_siswa, s.nama
ORDER BY s.nama;
```

`COUNT(n.id_nilai)` menghasilkan nol untuk siswa tanpa nilai. `COUNT(*)` pada LEFT JOIN dapat menghitung satu baris hasil join meski tidak ada nilai. `AVG()` menghasilkan NULL jika tidak ada nilai; jangan otomatis menggantinya menjadi nol karena maknanya berbeda.

12.3 Filter semester pada LEFT JOIN

```
SELECT s.id_siswa, s.nama, AVG(n.nilai_angka) AS rata_rata
FROM siswa s
LEFT JOIN nilai n ON n.id_siswa = s.id_siswa AND n.semester = 1
GROUP BY s.id_siswa, s.nama;
```

Filter ditempatkan di `ON` agar siswa yang belum memiliki nilai semester 1 tetap muncul. Jika ditempatkan di `WHERE n.semester = 1`, baris tanpa pasangan akan tersaring.

13. Transaksi untuk penyimpanan nilai sekaligus

Satu INSERT biasanya sudah atomik. Transaksi diperlukan saat beberapa perubahan harus berhasil bersama atau gagal bersama.

Latihan fungsi terpisah, dapat disimpan sebagai `admin\transaksi.php`. File ini hanya mendefinisikan fungsi; jangan dibuat endpoint publik yang langsung menyimpan tanpa POST, CSRF, dan otorisasi.

```
<?php
declare(strict_types=1);

function simpanNilaiSekaligus(PDO $pdo, DataRepository $repo, array $daftar): int
{
    if ($daftar === [] || count($daftar) > 100) {
        throw new InvalidArgumentException('Jumlah baris harus 1–100.');
    }
    $bersih = [];
    foreach ($daftar as $input) {
        if (!is_array($input)) {
            throw new InvalidArgumentException('Setiap baris harus berupa array input.');
        }
        $bersih[] = validasiData('nilai', $input, $repo);
    }
    $pdo->beginTransaction();
```

```
    try {
        foreach ($bersih as $data) {
            $repo->simpan('nilai', null, $data);
        }
        $pdo->commit();
        return count($bersih);
    } catch (Throwable $e) {
        if ($pdo->inTransaction()) {
            $pdo->rollBack();
        }
        throw $e;
    }
}
```

Repository harus memakai objek PDO yang sama dengan transaksi. Jangan memulai transaksi baru di dalam transaksi aktif tanpa rancangan savepoint. Validasi dilakukan sebelum transaksi untuk memperpendek waktu penguncian; constraint database tetap menjaga relasi saat INSERT.

Skenario pengujian transaksi

1. Siapkan dua kombinasi nilai yang belum ada.
2. Panggil fungsi dengan dua baris valid; jumlah data bertambah dua.
3. Siapkan batch baru dengan baris kedua menduplikasi kombinasi yang sudah ada.
4. Tangkap exception pada pemanggil.
5. Pastikan baris pertama batch kedua juga tidak tersimpan.
6. Jangan menjalankan DDL seperti `ALTER TABLE` di tengah pengujian transaksi DML.

14. Performa dan batas desain

- Pagination menghindari menampilkan seluruh baris sekaligus, tetapi COUNT tetap dapat mahal pada data sangat besar.
- `LIKE '%kata%'` biasanya tidak memanfaatkan indeks B-tree seperti pencarian prefix; ukur dengan `EXPLAIN` sebelum mengoptimalkan.
- Indeks komposit unik membantu pencarian yang menggunakan awalan kolom indeks, bukan semua kombinasi filter secara otomatis.
- Panel ini mengambil daftar pilihan induk sekali per request, bukan sekali per baris. Namun seluruh dropdown tetap dapat besar.
- Dua operator dapat saling menimpa perubahan karena contoh memakai last-write-wins. Pengembangan lanjutan: optimistic locking dengan kolom versi dan pemeriksaan jumlah baris update.
- Perubahan bersamaan antara pengecekan keberadaan dan DELETE dapat membuat operasi kedua tidak menghapus apa pun. Untuk audit ketat, tambahkan transaksi dan pemeriksaan hasil sesuai kebutuhan.

15. Matriks pengujian penerimaan

Kasus	Hasil yang diharapkan
Daftar empat entitas	Tampil tanpa error SQL
Tambah/edit data sah	Redirect ke daftar, data tersimpan
Refresh setelah simpan	Tidak mengulang INSERT
GET ke aksi hapus	HTTP 405, data tidak berubah
POST token kosong/salah	HTTP 403, data tidak berubah
ID berupa array/teks/negatif	Ditolak, tidak dipakai mentah dalam SQL
ID tidak ada	HTTP 404
Email/nama duplikat	Pesan konflik, tidak ada data ganda
Nilai ganda	Constraint unik menolak
Input nama mengandung `<` dan `>`	Ditampilkan sebagai teks, bukan HTML
Hapus siswa/mapel latihan	Nilai terkait ikut terhapus sesuai FK
Hapus jurusan latihan	Siswa tetap ada, jurusan NULL
Pencarian tanpa hasil	Pesan data tidak ditemukan
Halaman terlalu besar	Diklem ke halaman terakhir
Database berhenti	Pesan umum, bukan detail kredensial

Catat input, langkah, hasil aktual, dan apakah sesuai harapan. Gunakan database salinan untuk pengujian destruktif.

16. Latihan dan petunjuk penyelesaian

Latihan A — filter jurusan

Tambahkan filter jurusan pada daftar siswa. Gunakan placeholder untuk nilainya, dan kondisi yang sama pada query hitung maupun daftar. Jika hanya daftar yang difilter, pagination akan salah.

Latihan B — daftar nilai dengan JOIN

Ganti pemetaan nama pada daftar nilai dengan satu query JOIN. Pertahankan parameter pencarian, pagination, dan nama alias yang konsisten. Bandingkan jumlah query sebelum dan sesudah perubahan.

Latihan C — ekspor CSV

Ekspor laporan nilai setelah memeriksa hak akses pada Modul 7. Gunakan `fputcsv()`, bukan menyambung string dengan koma. Pertimbangkan formula injection saat CSV dibuka spreadsheet: nilai yang diawali `=`, `+`, `-`, atau `@`

memerlukan kebijakan penanganan yang sesuai. Jangan menaruh ekspor berisi data pribadi di folder publik tanpa kontrol akses.

Latihan D — pengujian batch

Bangun form batch maksimal 100 baris yang memanggil `simpanNilaiSekaligus()`. Terapkan POST, CSRF, role, validasi setiap baris, dan pesan yang menyebut baris bermasalah. Uji rollback ketika baris terakhir gagal.

17. Troubleshooting

Gejala	Pemeriksaan
`Undefined variable $pdo`	Pastikan koneksi Modul 4 mendefinisikan `$pdo`
Table/column not found	Cocokkan skema DDL, bukan skema dari tutorial lain
Error 1062	Email, nama, atau kombinasi nilai duplikat
Error 1452	ID siswa/mapel/jurusan tidak tersedia
Token selalu gagal	Cookie session aktif, bootstrap sebelum output, form memuat token
Redirect gagal	Tidak ada spasi/BOM/output sebelum `<?php`
Halaman kosong	Lihat log PHP/Apache; jangan hanya menyalakan error publik
Nilai `75,5` ditolak	Kontrak input menggunakan titik desimal
UNIQUE gagal ditambahkan	Selesaikan duplikat lama dahulu

18. Rubrik dan kelanjutan

Aspek	Bobot
CRUD seluruh entitas	30
Validasi dan constraint	25
POST, CSRF, escape output	20
Relasi, laporan, transaksi	15
Dokumentasi pengujian	10

Sebelum lanjut, pastikan semua kasus utama lulus. Berikutnya Modul 7 menambahkan autentikasi ke panel ini **dan halaman lama Modul 4**, agar tidak ada jalur akses yang tertinggal.

19. Referensi

- PHP PDO

- PDO prepared statements
- PDO transactions
- MySQL foreign keys